

Slices - Cheat Sheet

Contents

1. [Slice header](#)
2. [Creating slices](#)
3. [append\(\) and growth](#)
4. [The aliasing trap](#)
5. [Slicing syntax](#)
6. [Length and capacity](#)
7. [Copy](#)
8. [nil slice vs empty slice](#)
9. [Iteration](#)
10. [Slice tricks](#)
11. [slices package \(Go 1.21+\)](#)
12. [Slices as function parameters](#)
13. [Slices vs arrays](#)
14. [Memory leak: slice keeping a big array alive](#)
15. [Best practices](#)
16. [Common gotchas](#)
17. [Quick reference](#)

A slice is Go's main collection type. Backed by an array, but resizable. The slice header itself is three words: a pointer to the underlying array, a length, and a capacity. Passing a slice around copies the 3-word header. The underlying data stays where it was.

The behavior that confuses Go newcomers most: two slices can share the same underlying array. Mutating one is visible to the other. Append can return a different array entirely. Understanding the header model fixes most of these surprises.

Slice header

A slice is just a 3-word struct.

```
type SliceHeader struct {  
    Data uintptr // pointer to the first element of the backing array  
    Len  int      // current length  
    Cap  int      // capacity of the backing array from Data onward  
}
```

`len(s)` returns `Len`. `cap(s)` returns `Cap`. The backing array isn't part of the slice. Multiple slices can point into the same array.

```
backing array: [ 10 | 20 | 30 | 40 | 50 ]
                ^       ^
                |       |
                s := arr[0:3] Len=3, Cap=5
                  t := arr[2:4] Len=2, Cap=3
```

`s` and `t` share memory. Modifying `s[2]` changes `t[0]`.

Creating slices

```
// Literal
s := []int{1, 2, 3} // len=3, cap=3

// make: len only (cap = len)
s := make([]int, 5) // len=5, cap=5, all zeros

// make: len + cap
s := make([]int, 0, 100) // len=0, cap=100. Empty but preallocated.

// From an array
arr := [5]int{1, 2, 3, 4, 5}
s := arr[1:4] // len=3, cap=4 (from index 1 to end of array)

// From another slice
s2 := s[0:2] // shares underlying array with s

// nil slice
var s []int // len=0, cap=0, no backing array
```

When you know the final size, use `make([]T, 0, n)` and `append`. Cuts allocations to one.

```
// Bad: each append may reallocate
s := []int{}
for i := 0; i < 1000; i++ {
    s = append(s, i)
}

// Good: one allocation
s := make([]int, 0, 1000)
for i := 0; i < 1000; i++ {
    s = append(s, i)
}
```

append() and growth

`append` adds elements. When `len == cap`, it allocates a bigger backing array, copies the existing data, and returns a slice with a new pointer.

```
s := []int{1, 2, 3}           // len=3, cap=3
s = append(s, 4)              // len=4, cap=6 (often). New array.
```

Growth strategy (current Go runtime, approximate):

- Small slices (under ~256 elements): roughly double when full.
- Larger slices: grow by ~25% to avoid huge over-allocation.

The exact numbers can change. The point: appending in a loop without preallocating triggers $\log(n)$ reallocations, each copying the existing elements.

`append` returns the (possibly new) slice. Always assign the return value:

```
s = append(s, 1, 2, 3)        // append multiple
s = append(s, other...)       // append a whole slice
```

The aliasing trap

If `len < cap`, `append` writes into the existing backing array. This is where many bugs live.

```
s := make([]int, 3, 5)        // [0, 0, 0], cap 5
s[0] = 1; s[1] = 2; s[2] = 3  // [1, 2, 3]

a := s[:2]                    // [1, 2], same backing array, cap 5
b := append(a, 99)             // [1, 2, 99], wrote into s[2]!

fmt.Println(s)                 // [1, 2, 99] -- s changed unexpectedly
```

`append(a, 99)` had room in the backing array (`cap(a) == 5`), so it didn't allocate. It wrote 99 into the slot that `s[2]` also points to.

Slicing syntax

```
s := []int{10, 20, 30, 40, 50}

s[1:4]      // [20, 30, 40]
s[:3]       // [10, 20, 30]
s[2:]       // [30, 40, 50]
s[:]        // [10, 20, 30, 40, 50]

// Three-index form: s[low:high:max]
// Sets capacity explicitly. cap = max - low.
s[1:3:3]     // [20, 30], cap=2 -- next append must allocate
```

Three-index slicing is the fix for the aliasing trap above. By setting `max == high`, you guarantee that any append will reallocate.

```
a := s[:2:2]
b := append(a, 99)           // safe: cap was 2, append allocates a new backing array
```

Use this when you return a slice from a function and want callers locked out of clobbering your data via `append`.

Length and capacity

```
s := make([]int, 3, 10)
len(s)    // 3
cap(s)    // 10
```

`len` is the visible size. `cap` is the size of the backing array from the slice's start.

For `t := s[1:3]`:

- `len(t)` is `3 - 1 = 2`.
- `cap(t)` is `cap(s) - 1`. The capacity loses elements only at the front.

This matters for `append`. If `cap(t)` is high, `append(t, ...)` may overwrite elements past `len(t)` that you didn't intend to touch.

Copy

```
src := []int{1, 2, 3, 4, 5}
dst := make([]int, 3)
n := copy(dst, src)           // n=3, dst=[1,2,3]
```

`copy` copies `min(len(dst), len(src))` elements. Returns how many were copied. Works between overlapping slices too.

When you need an independent slice (no shared backing array):

```
clone := make([]int, len(src))
copy(clone, src)

// Or use slices.Clone (Go 1.21+)
clone := slices.Clone(src)
```

nil slice vs empty slice

Three things look similar but differ:

```
var a []int           // nil: len=0, cap=0, ptr=nil
b := []int{}          // empty: len=0, cap=0, ptr=non-nil
c := make([]int, 0)    // empty: len=0, cap=0, ptr=non-nil
```

For most code they behave identically:

- `len`, `cap`, `range`, `append` all work on nil slices.
- `append(nil_slice, 1)` returns a non-nil slice with one element.

Where they differ:

- `a == nil` is `true`. `b == nil` and `c == nil` are `false`.
- JSON encoding: nil slice becomes `null`, empty slice becomes `[]`.

Prefer the nil form unless you specifically need a non-nil empty slice (often for JSON shape).

Iteration

```
s := []string{"a", "b", "c"}

// Index + value
for i, v := range s {
    fmt.Println(i, v)
}

// Index only
for i := range s {
    fmt.Println(i)
}

// Value only
for _, v := range s {
    fmt.Println(v)
}

// Classic for
for i := 0; i < len(s); i++ {
    fmt.Println(s[i])
}
```

`range` evaluates the slice header once at the start. Modifying the slice during iteration (via `append` or slicing) doesn't change the loop's view.

```
s := []int{1, 2, 3}
for i, v := range s {
    s = append(s, v * 10)    // doesn't extend the loop
    fmt.Println(i, v)       // prints (0,1), (1,2), (2,3) only
}
```

Each iteration in Go 1.22+ gets its own `v`. Pre-1.22, `v` was reused, which caused goroutine-capture bugs.

Slice tricks

Standard idioms from the Go wiki.

Pop front

```
x, s := s[0], s[1:]
```

`s` no longer references element 0 directly, but the backing array still does. The first element can't be garbage collected until `s` is resliced past it or replaced.

Pop back

```
x, s := s[len(s)-1], s[:len(s)-1]
```

Insert at index i

```
s = append(s[:i], append([]T{v}, s[i:]...))...  
  
// Cleaner with slices.Insert (Go 1.21+)  
s = slices.Insert(s, i, v)
```

Delete at index i (preserve order)

```
s = append(s[:i], s[i+1:]...)  
  
// Or  
s = slices.Delete(s, i, i+1)
```

Delete at index i (fast, swap with last)

```
s[i] = s[len(s)-1]  
s = s[:len(s)-1]
```

O(1), but reorders the slice.

Filter in place

```
n := 0  
for _, v := range s {  
    if keep(v) {  
        s[n] = v  
        n++  
    }  
}  
s = s[:n]
```

No allocation. The original elements past `n` may still be referenced by the backing array, so for pointer types, zero them out:

```
for i := n; i < len(s); i++ {  
    s[i] = nil  
}  
s = s[:n]
```

Reverse

```

for i, j := 0, len(s)-1; i < j; i, j = i+1, j-1 {
    s[i], s[j] = s[j], s[i]
}

// Or
slices.Reverse(s)

```

slices package (Go 1.21+)

The slices package adds generic helpers:

```

import "slices"

slices.Contains([]int{1,2,3}, 2)           // true
slices.Index([]int{1,2,3}, 2)             // 1
slices.Max([]int{3,1,2})                  // 3
slices.Min(...)
slices.Sort([]int{3,1,2})                  // mutates in place
slices.SortFunc(s, func(a, b T) int { return cmp(a, b) })
slices.IsSorted(s)                        // bool
slices.BinarySearch(s, v)
slices.Reverse(s)
slices.Clone(s)
slices.Equal(s1, s2)                      // element-wise equality
slices.Concat(s1, s2, s3)
slices.Delete(s, i, j)
slices.Insert(s, i, vs...)
slices.Compact(s)                         // remove consecutive duplicates

```

Replaces a lot of hand-written loops. Type-safe via generics.

Slices as function parameters

Slices pass by value, but the value is the 3-word header. The underlying array is shared.

```

func zero(s []int) {
    for i := range s {
        s[i] = 0
    }
}

s := []int{1, 2, 3}
zero(s)
fmt.Println(s)           // [0, 0, 0] -- mutated in place

```

A function can't make the caller's slice longer just by appending. The header is a copy.


```
func bad(s []int) {
    s = append(s, 99)           // caller doesn't see this if it caused reallocation
}
```

To grow the slice and have the caller see it, return the new slice:

```
func add(s []int, v int) []int {
    return append(s, v)
}

s = add(s, 99)
```

Or take a `*[]int` if you must mutate in place. Returning is the idiomatic form.

Slices vs arrays

Aspect	Array	Slice
Syntax	<code>[5]int</code>	<code>[]int</code>
Size	Fixed at compile time	Dynamic
Pass by	Value (copies all elements)	Value of header (shares array)
Comparable	Yes (<code>==</code> works)	No (compile error)
Common usage	Rare in app code	Default for sequences
Zero value	All zero elements	nil (no backing array)

Arrays show up in stdlib internals and fixed-size buffers (`[32]byte` for a hash). Application code is almost always slices.

Memory leak: slice keeping a big array alive

```
func first10(b []byte) []byte {
    return b[:10]
}

// Caller:
huge := readBigFile()           // 100 MB
small := first10(huge)
huge = nil                       // doesn't help. `small` still references the 100 MB array.
```

The returned slice keeps the backing array alive. GC can't free the 100 MB until `small` is gone.

Fix: copy out the slice you want to keep.

```
small := make([]byte, 10)
copy(small, huge[:10])
huge = nil    // now the 100 MB can be GC'd
```

Or use `slices.Clone`:

```
small := slices.Clone(huge[:10])
```

Best practices

- Preallocate when you know the size. `make([]T, 0, n)` saves $N \log N$ allocations and copies.
- Always assign `append`'s return value. `s = append(s, x)`.
- Use the three-index slice (`s[a:b:b]`) when returning a sub-slice from a function to prevent callers from clobbering your data via `append`.
- `copy` for independent slices. Plain assignment (`s2 := s1`) shares the backing array.
- `slices.Clone` for a defensive copy (Go 1.21+). Replaces the `make` + `copy` pattern.
- `nil` slice is fine for most use cases. Reach for `[]T{}` only when JSON encoding demands it.
- Zero out pointer slots before truncating if the elements are pointers or contain pointers. Otherwise the GC can't reclaim what's past `len`.
- `go vet` and the race detector catch most slice-aliasing bugs.

Common gotchas

- Append aliasing. `append(s[:n], x)` may overwrite elements of the parent slice if `cap(s) > n`. Three-index slicing fixes it.
- Slicing keeps the backing array alive. Returning a small slice from a big buffer holds the whole buffer in memory.
- Modifying a slice during iteration. The loop sees the original `len`. New elements appended in the body aren't iterated.
- Comparing slices with `==`. Compile error. Use `slices.Equal` (Go 1.21+) or `reflect.DeepEqual`.
- `copy(dst, src)` does the minimum of the two lengths. If `dst` is empty, it copies nothing. Pre-size `dst`.
- Nil slice JSON encoding. Encodes as `null`. If your API needs `[]`, use a non-nil empty slice.
- Range value reuse (pre-Go 1.22). `for _, v := range s { go use(&v) }` captures the same variable across iterations. Fixed in 1.22.
- `make([]T, 0, n)` vs `make([]T, n)`. The second one creates `n` zero elements. The first creates capacity but no elements. Subsequent `append` extends from `len 0`.
- Casting between `[]byte` and `string`. Copies the data both ways. For zero-copy conversions, use `unsafe` only when you've measured and need the win.
- `s[i] = nil` doesn't shrink the slice. It just zeros the element. Use `s = s[:len(s)-1]` to shrink.

- `append(nil, ...)` works. A nil slice is a valid receiver for append. It allocates the first backing array.

Quick reference

Operation	Code
Literal	<code>[]int{1, 2, 3}</code>
Empty preallocated	<code>make([]int, 0, n)</code>
Length	<code>len(s)</code>
Capacity	<code>cap(s)</code>
Append	<code>s = append(s, x)</code>
Append a slice	<code>s = append(s, other...)</code>
Copy	<code>copy(dst, src)</code> returns count copied
Sub-slice	<code>s[low:high]</code>
Sub-slice with cap	<code>s[low:high:max]</code>
Pop front	<code>x, s = s[0], s[1:]</code>
Pop back	<code>x, s = s[len(s)-1], s[:len(s)-1]</code>
Insert at i	<code>slices.Insert(s, i, v)</code>
Delete at i	<code>slices.Delete(s, i, i+1)</code>
Reverse	<code>slices.Reverse(s)</code>
Sort	<code>slices.Sort(s)</code> or <code>slices.SortFunc(...)</code>
Clone	<code>slices.Clone(s)</code>
Contains	<code>slices.Contains(s, v)</code>
nil check	<code>s == nil</code>
Range	<code>for i, v := range s { ... }</code>